Grip, Trace, Write: Force-Aware Imitation and Reinforcement Learning for High-DoF Robotic Handwriting

Rongqian Chen **Yiding Chen** **Jijie Zhou** **Tom Z. Jiahao**

Daxo Robotics
https://dexwriting.github.io/

Abstract: Most robotic handwriting rigidly mounts the pen and reduces writing to trajectory tracking. A freely held pen instead couples grasp stability, tip motion, and contact force. We present *Grip, Trace, Write*, a force-aware simulation environment and learning framework for high-DoF handwriting. A two-branch actor separates pen retention from arm–wrist writing while preserving their closed-loop coupling. Training learns a grasp controller, initializes writing from inverse-kinematics demonstrations by behavior cloning, and improves writing through reinforcement learning with rewards on ink, force, time, and retention. Across diverse tasks, the policy improves completion, precision, force regulation, and efficiency over behavior cloning and inverse kinematics, and generalizes without fine-tuning to unseen English and Chinese content. These results establish force-aware handwriting as a testbed for dexterous tool use.

Keywords: Dexterous Manipulation, Force-Aware Control, Imitation Learning, Reinforcement Learning

1 Introduction

Writing with a freely held pen is a compact testbed for dexterous tool use. It requires a hand to stabilize a slender object through a precision grip while the arm and wrist transport its tip and regulate its contact with a surface. Human manual dexterity similarly combines stable multi-finger grasping, fine motion, and force-sensitive interaction with tools [1].

This coupling makes handwriting fundamentally different from moving a rigidly mounted pen along a geometric path. A desired tip trajectory alone cannot determine whether ink is deposited: insufficient force produces hovering or broken strokes, whereas excessive force can distort the trace, alter the pen pose, or destabilize the grasp. With a freely held pen, finger–pen contact, pen–sheet contact, and arm motion therefore affect one another. Existing systems commonly avoid this coupling by rigidly mounting the pen or brush to an arm [2, 3, 4, 5], or address only limited writing behaviors with multi-fingered hands [6, 7]. They typically treat writing as trajectory tracking [8], with contact force uncontrolled or maintained around a fixed setpoint, and evaluate only a small set of strokes or glyphs [9, 7]. Consequently, sustained force-regulated writing with a freely held pen and a high-DoF hand remains unresolved.

We address this problem in a force-aware simulation environment for robotic handwriting. A learned grasp MLP first establishes a stable hand–pen interface. Given a requested 5D cursor, an inverse-kinematics (IK) controller generates nominal arm–wrist commands while the learned grasp holds the pen; these rollouts initialize a trajectory-conditioned writing policy through behavior cloning (BC). BC provides broad coverage of the writing repertoire but only imitates nominal IK actions.

We therefore improve writing through RL with rewards defined on task outcomes: ordered ink coverage, off-target ink, force tracking, elapsed time, and pen retention. This separation makes the roles explicit: grasp learning resolves tool stabilization, IK-guided BC makes high-DoF writing tractable, and RL corrects contact-induced errors that action imitation does not optimize.

Our contributions are: (1) a force-aware simulation environment for high-DoF robotic handwriting with a freely held pen, contact dynamics, and diverse tracing and writing tasks; (2) a two-branch controller that separates pen retention from arm–wrist writing while retaining their closed-loop physical coupling; and (3) a grasp-to-writing learning procedure that uses IK-guided BC for broad initialization and task-outcome RL to improve trace completion, ink precision, contact-force accuracy, and execution efficiency.

Related work. Prior robot-writing and drawing systems typically rigidly mount a pen or brush to an arm, reducing the problem to end-effector trajectory generation or ink modeling [2, 3, 4, 5]. Writing with a pen held in a multi-fingered hand remains rare and limited to single strokes or simple shapes, using synergies [6], online Jacobian estimation [8], or tactile RL with compliant wrist control [7]. Learned in-hand policies reorient and spin objects without external contact [10, 11, 12], track a held stick's tip without touching a surface [9], or bring a held tool into contact without regulating force [13]. Initializing from demonstrations and refining with RL is effective for dexterous control [14, 15]; we apply it to the full writing repertoire with outcome rewards on ink, force, and pen retention.

2 Method

2.1 Environment and control setup

Our MuJoCo setup [16] mounts a high-DoF anthropomorphic hand (Fig. 1(a)(b)) on a UR5e and places a free pen in a tripod grasp without virtual assistance. The hand has 25 finger joints, in addition to 2 wrist and 7 arm joints. The suite combines three retention conditions (static, moving, and contact scribbling), three tracing families (line, circle, and varied shapes), and handwriting of digits, capital letters, English words, and Chinese characters. Training covers all 10 digits, 26 capital letters, 15 English words, and 118 Chinese characters with several fonts. Every episode resamples placement, scale, slant, rotation, and target-force profile. Zero-shot tests add 20 held-out English words, 10 four-character Chinese idioms, and 122 Chinese characters with at least seven strokes. Appendix B gives complete task and interface definitions.

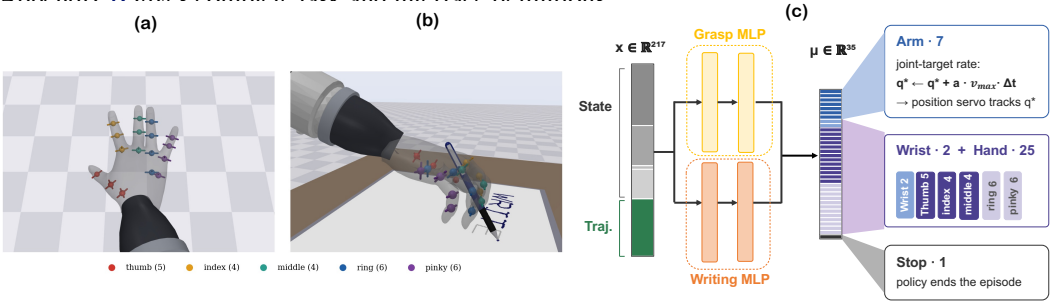


Figure 1: **Hand kinematics and two-branch actor.** (a)(b) Spheres mark the finger joints, colored by finger, with rods along the hinge axes; the skin is rendered translucent. The hand has 25 finger joints, in addition to 2 wrist and 7 arm joints. (c) Actor model reads both the same state and trajectory. A fixed output mask selects finger actions from the grasp branch and UR5e arm, wrist, and stop actions from the writing branch. The branches interact through the held pen and live state.

2.2 Two-branch policy

The actor has separate grasp and writing MLPs (Fig. 1(c)). A fixed mask takes finger actions from the grasp trunk and arm, wrist, and stop actions from the writing trunk. This split follows the

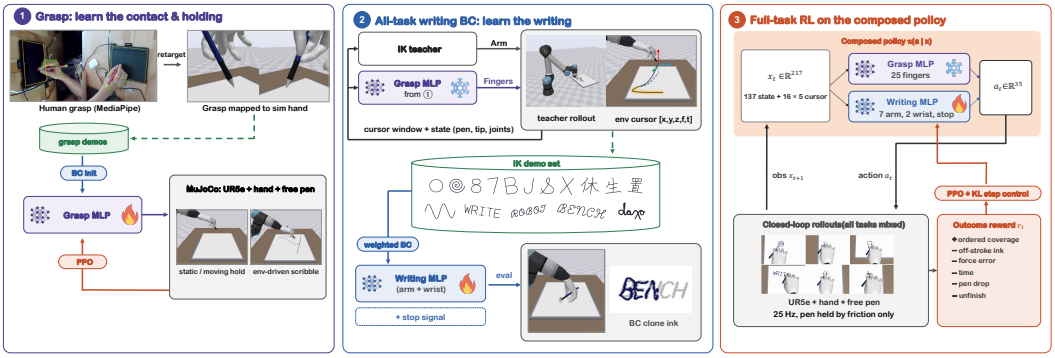


Figure 2: **Training pipeline.** Panels 1–3 correspond to the three stages in Sec. 2.3: grasp BC and hold RL, all-task writing BC with the learned grasp, and full-task outcome RL on the composed policy.

structure of the problem: pen retention can be learned from holding interactions before any glyph is specified, whereas tip transport can use IK once a stable grasp exists. Both trunks still read the same live state, so the mask separates supervision and action roles without breaking the physical coupling among hand, pen, and arm. The transferred grasp controller is folded into the new input representation; its preservation during final RL is described below.

2.3 Training workflow

Grasp BC and RL. Stable pen retention is a prerequisite for meaningful handwriting learning. When the pen slips, missing or misplaced ink can arise from either failed finger contact or incorrect arm motion, so the writing objective does not clearly identify which subsystem should change. We therefore learn the grasp branch before the writer. BC on tripod-hold demonstrations quickly initializes a valid multi-finger contact configuration; pen-hold RL then improves retention under static, arm-driven, and environment-driven motion, covering contact disturbances absent from the demonstrations. Transferring this controller gives subsequent IK data collection and writing optimization a consistent, physically stable hand–pen interface.

All-task writing BC. Once the grasp is reliable, the remaining problem is to map a requested trajectory to arm and wrist motion under that particular hand–pen contact. We therefore run IK while the transferred grasp controls the fingers. This produces demonstrations in the same compliant, freely held system that the learned writer will encounter, avoiding a mismatch with data collected using an idealized fixed pen. Clean rollouts teach nominal motion; perturbing the executed action moves the system into nearby off-nominal states while retaining the clean IK command as the recovery target [17]. Because traces, glyphs, words, and Chinese writing are all presented through the same trajectory interface, we pool them to learn a geometry-conditioned controller rather than separate content-specific policies. The writing branch minimizes

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}_{(x,a^*)} \sum_{d=1}^{9} w_d \big(\mu_d(x) - a_d^*/s_d\big)^2, \quad w_d = \mathrm{clip}\left[\left(\frac{\mathrm{median}_j\, \sigma_j}{\max(\sigma_d, 10^{-3})}\right)^2, 1, 100\right], \tag{1}$$

where s_d scales action dimension d and σ_d is its label standard deviation. Inverse-variance weighting balances coordinates with different natural ranges, and fitting the stop output on the clone's own rollouts teaches termination under its induced state distribution. This stage gives the complete system a broad, stable writing initialization, but it still optimizes agreement with IK rather than the physical trace.

Full-task outcome RL. BC solves initialization, but leaves two gaps: matching IK commands does not directly optimize ink and force, and small execution errors can carry the clone beyond the demonstrated states. We therefore refine the composed controller with PPO on the complete

task mixture. The grasp mean remains frozen so drawing reward cannot trade away the reliable contact strategy, and frozen observation statistics prevent normalization drift from changing the transferred policy. The writing mean and Gaussian exploration parameters remain trainable, while both branches stay closed-loop through their shared state and the physical pen. Let C_t be ordered coverage within 1 mm, P_t normalized off-target ink, E_t running force MAE, τ_t normalized elapsed time, p_t the pen tip, and q_t the first unfinished target point at pressing depth. The outcome reward is

$$r_t = \lambda_C \Delta C_t - \lambda_P \Delta P_t - \lambda_E \Delta E_t - \lambda_\tau \Delta \tau_t + \lambda_q \big(\|p_{t-1} - q_t\| - \|p_t - q_t\| \big) - D_t, \qquad (2)$$

where all λ weights are positive and D_t penalizes drops and premature termination. Each term targets a distinct failure mode: ordered coverage rewards forward stroke progress, off-target ink rejects indiscriminate scribbling, and the pressing-depth point q_t prevents progress by hovering above the sheet. Force and time distinguish meaningful contact from mere touching and favor efficient completion, while D_t preserves the pen-retention prerequisite established in the first stage. Crucially, there is no BC or IK-timing term, so IK provides an initialization rather than a performance ceiling. Appendix A.2 gives the numerical weights, penalty construction, PPO schedule, and optimization details.

3 Experiments

Protocol. We compare IK, all-task BC, and final RL on eight primary tasks with 50 matched seeds each. Policies execute deterministic means and their own stop outputs. We report ordered completion and precision within 1 mm, force MAE, time, shape distances, and pen drops; generalization tests use unseen English and Chinese content without fine-tuning.

3.1 Writing performance

Controller	Metric	Primary tasks								Unseen			Mean
		Line	Circle	Shape	Digits	Letters	Word	Cursive	Chinese	word	Idiom	Chinese	
IK	Completion (%) ↑	**98.2**	98.8	94.9	90.6	92.7	85.4	83.4	91.7	87.7	92.1	87.7	92.0
	Precision (%) ↑	95.7	96.4	82.6	75.8	82.2	71.5	61.6	78.7	71.1	82.1	71.7	80.6
	Force MAE (N) ↓	0.558	0.537	0.529	0.531	0.533	0.582	0.526	0.497	0.585	0.580	0.524	0.537
	Time (s) ↓	3.16	**4.47**	7.38	5.05	5.91	22.27	8.48	8.19	18.96	**27.71**	12.44	8.11
BC	Completion (%) ↑	96.7	97.6	93.6	90.2	92.5	85.7	82.4	91.5	87.2	91.6	86.9	91.3
	Precision (%) ↑	93.4	93.6	81.1	74.2	80.8	71.4	60.6	77.0	70.2	79.8	70.3	79.0
	Force MAE (N) ↓	0.590	0.763	0.707	0.866	0.801	0.943	0.753	0.764	0.792	0.605	0.856	0.773
	Time (s) ↓	**3.10**	4.54	7.26	5.26	6.01	22.31	8.59	8.45	18.84	28.48	12.52	8.19
RL	Completion (%) ↑	97.5	**99.7**	**97.1**	**95.0**	**95.8**	**89.8**	**86.8**	**96.4**	**91.6**	**92.9**	**91.6**	**94.8**
	Precision (%) ↑	**96.6**	**99.0**	**90.6**	**86.5**	**89.3**	**80.3**	**69.6**	**89.3**	**80.9**	**89.4**	**83.0**	**87.6**
	Force MAE (N) ↓	**0.310**	**0.325**	**0.369**	**0.348**	**0.380**	**0.438**	**0.396**	**0.331**	**0.433**	**0.454**	**0.402**	**0.362**
	Time (s) ↓	3.13	4.73	**7.09**	**4.66**	**5.65**	**20.58**	**7.72**	**7.78**	**17.48**	27.84	**11.28**	**7.67**

Table 1: Task-level and zero-shot results with 50 matched seeds per task. Completion, precision, force MAE, and time are reported on separate rows. The mean covers the eight primary tasks; the three unseen columns are zero-shot evaluations. Bold marks the best controller for each task and metric.

Table 1 shows that RL raises mean completion to 94.8%, from 91.3% for BC and 92.0% for IK. BC itself is below IK in both completion and precision, confirming its role as an initialization rather than the final controller. Relative to BC, RL raises precision from 79.0% to 87.6%, reduces force MAE by 53% from 0.773 to 0.362 N, and reduces time from 8.19 to 7.67 s. The IK and BC traces exhibit visible stroke jitter, which leaves ink outside the target and depresses precision; outcome RL suppresses this effect (Appendix Fig. 6). RL improves completion on seven of eight primary tasks and precision and force error on all eight. Its gains on unseen English words, Chinese idioms, and Chinese characters show execution of new requested trajectories rather than memorization. Appendix C provides complete metrics and diagnostics.

4 Conclusion

We present a force-aware grasp-to-writing system. A transferred grasp trunk retains the pen, all-task IK-guided BC initializes writing, and outcome RL refines the controller. RL improves completion, precision, force, and time over BC and IK, including on unseen English and Chinese content.

References

[1] T. L. Kivell, N. Baraki, V. Lockwood, E. M. Williams-Hatala, and B. A. Wood. Form, function and evolution of the human hand. *American Journal of Biological Anthropology*, 181:6–57, 2023.

[2] S. Mueller, N. Huebel, M. Waibel, and R. D'Andrea. Robotic calligraphy—learning how to write single strokes of Chinese and Japanese characters. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1734–1739. IEEE, 2013. doi:10.1109/IROS.2013.6696583.

[3] R. Wu, W. Fang, F. Chao, X. Gao, C. Zhou, L. Yang, C.-M. Lin, and C. Shang. Towards deep reinforcement learning based Chinese calligraphy robot. In *IEEE International Conference on Robotics and Biomimetics*, pages 507–512. IEEE, 2018. doi:10.1109/ROBIO.2018.8664813.

[4] Y. Luo, Z. Wu, and Z. Lian. CalliRewrite: Recovering handwriting behaviors from calligraphy images without supervision. In *IEEE International Conference on Robotics and Automation*, pages 8671–8678. IEEE, 2024. doi:10.1109/ICRA57147.2024.10610332.

[5] P. Schaldenbrand, J. McCann, and J. Oh. FRIDA: A collaborative robot painter with a differentiable, real2sim2real planning environment. In *IEEE International Conference on Robotics and Automation*, pages 11712–11718. IEEE, 2022. doi:10.1109/ICRA48891.2023.10160702.

[6] E. Rombokas, M. Malhotra, and Y. Matsuoka. Task-specific demonstration and practiced synergies for writing with the ACT hand. In *IEEE International Conference on Robotics and Automation*, pages 5363–5368. IEEE, 2011. doi:10.1109/ICRA.2011.5980218.

[7] C. Zhao, L. Xie, B. Huang, S. Wang, and D. Ma. Tactile-driven dexterous in-hand writing via extrinsic contact sensing. *IEEE Robotics and Automation Letters*, 10(9):8914–8921, 2025. doi:10.1109/LRA.2025.3589168.

[8] K. Stewart, Y. Toshimitsu, and R. K. Katzschmann. Rapid learning of dexterous in-hand pen writing through real-time Jacobian estimation. *arXiv*, 2026. URL `https://arxiv.org/abs/2609.11775`.

[9] W. Hu, B. Huang, W. W. Lee, S. Yang, Y. Zheng, and Z. Li. Dexterous in-hand manipulation of slender cylindrical objects through deep reinforcement learning with tactile sensing. *Robotics Auton. Syst.*, 186:104904, 2023. doi:10.2139/ssrn.4674022.

[10] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

[11] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. J. Liu, D. Makoviichuk, K. V. Wyk, A. Zhurkevich, B. Sundaralingam, et al. DeXtreme: Transfer of agile in-hand manipulation from simulation to reality. In *IEEE International Conference on Robotics and Automation*, pages 5977–5984. IEEE, 2022. doi:10.1109/ICRA48891.2023.10160216. arXiv:2210.13702.

[12] J. Wang, Y. Yuan, H. Che, H. Qi, Y. Ma, J. Malik, and X. Wang. Lessons from learning to spin "pens". In *Conference on Robot Learning*, 2024. doi:10.48550/arXiv.2407.18902. arXiv:2407.18902.

[13] K. Kedia, T. Lum, J. Bohg, and C. K. Liu. SimToolReal: An object-centric policy for zero-shot dexterous tool manipulation. In *Robotics: Science and Systems XXII*. Robotics: Science and Systems Foundation, 2026. doi:10.15607/RSS.2026.XXII.151.

[14] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems Conference*. Robotics: Science and Systems Foundation, 2017. doi:10.15607/RSS.2018.XIV.049. arXiv:1709.10087.

[15] Y. Qin, Y.-H. Wu, S. Liu, H. Jiang, R. Yang, Y. Fu, and X. Wang. Dexmv: Imitation learning for dexterous manipulation from human videos. In *European Conference on Computer Vision*, pages 570–587. Springer, 2022.

[16] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[17] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg. DART: Noise injection for robust imitation learning. In *Conference on Robot Learning (CoRL)*, 2017. URL https://proceedings.mlr.press/v78/laskey17a.html.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

A Implementation and evaluation details

Reading the supplementary material. This appendix provides implementation details, task definitions, training settings, ablations, and extended results for the reported trajectory-conditioned system.

Figure 3 shows the simulated scene. Hand and environment settings are listed in Table 2; network, training, and evaluation settings are listed separately in Table 3. The physical control contract is therefore distinguished from choices made by the learning algorithm.

Parameter	Hand and environment configuration
Hand and arm	High-DoF anthropomorphic robotic hand mounted on a UR5e assembly
Actuated coordinates	Seven arm coordinates, two wrist joints, and 25 finger joints
Physical action interface	Seven integrated arm-position increments; 27 speed–torque-limited wrist/finger duties
Control frequency	25 Hz
Tool and initial grasp	Free rigid-body pen, initially seated in a tripod grasp; retained through contact
Grip assistance	No virtual grip assistance in the reported writing configuration
Starting height	2–6 mm in the reported writing configuration
Trajectory advancement	Tip-driven pointer; pen-down proximity tolerance 1.5 mm; skip timeout 0.25 s
Task set	Line, circle, shape, digits, letters, word, cursive, and Chinese writing
Content split	Shared training/evaluation content pool; primary numerical comparison uses the eight-task suite

Table 2: Hand, environment, and task-interface settings for the reported writing study. These specify the physical control problem; model architecture and optimization settings are given separately in Table 3.

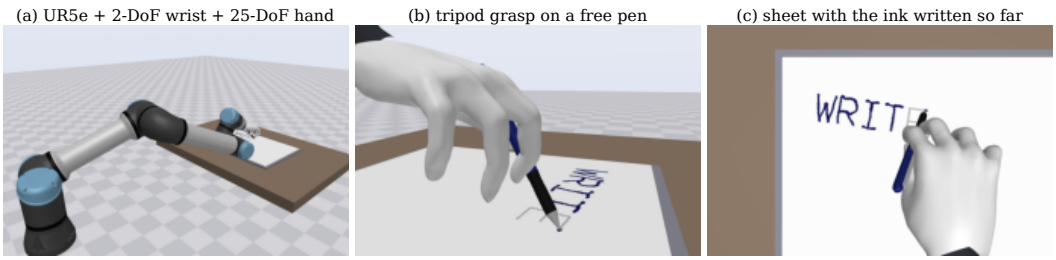


Figure 3: Simulated setup. (a) A UR5e arm with a 2-DoF wrist and a 25-DoF anthropomorphic hand writes on paper with a free pen. (b) The tripod grasp. (c) Top view mid-episode; gray marks the remaining target.

Parameter	Model, training, and evaluation configuration
Model architecture and observation processing	
Input representation	137 state entries plus 16 trajectory samples of five entries each; samples every two trajectory indices; no image encoder
Actor	Separate grasp and writing MLPs, each $217 \to 256 \to 256$ with tanh activations; masked linear heads
Policy output	34 physical-action entries plus one learned stop output; stop threshold 0.5
Critic	Independent $217 \to 256 \to 256 \to 1$ MLP
Normalization	Vector statistics frozen during final RL; normalized observations clipped to $[-10, 10]$
Behavior cloning	
Demonstration noise	Clean and noisy demonstrations, $\sigma = 0.01, 0.02$
BC objective	Arm/wrist loss with inverse-variance weighting
BC optimizer	Adam, learning rate 3×10^{-4}, batch 256, 60 epochs per candidate
Reinforcement learning	
Grasp preservation	Grasp-mean parameters frozen; Gaussian log-standard-deviations remain trainable
Rollout collection	512 environments $\times$ 1024 control steps; asynchronous execution
Actor update	PPO clip 0.2; full-rollout gradient accumulation; up to four passes; approximate KL target 0.03 on 16,384 randomly sampled transitions
Optimizer and returns	Adam 10^{-5} with linear decay; minibatch 2048; $\gamma = 1$, GAE $\lambda = 0.95$; reward scale 0.001
Critic update	Three initial critic-only iterations, followed by two critic passes per rollout
Initial exploration	Log-standard-deviations: arm/wrist -4.6, fingers -3.9, stop -3.0; floor -6.0
Recorded budget	39,845,888 transitions; 193.8 minutes for the reported final RL run
Evaluation protocol	
Policy execution	Deterministic learned mean and the network's own stop output
Evaluation sample	Primary comparison: 50 seeds $\times$ eight tasks

Table 3: Model and learning settings, separated from the hand/environment parameters in Table 2. BC settings describe the available collection/training recipe, not a reconstructed per-shard manifest of the final clone. Evaluation settings specify measurement rather than training updates.

Data and checkpoint provenance. The main comparison uses an eight-task evaluation with 50 episodes with different seeds per task for IK, all-task BC, and final RL. Intermediate evaluations in the learning plot are snapshots of the same final RL run.

Evaluation-suite changes. The development code merged the original content pools, and new evaluation-only word, idiom, and Chinese-character pools were constructed for the zero-shot tests. A later script-font update and the folding of cursive into the word task change the distribution relative to the primary eight-task evaluation.

A.1 Training and checkpoint details

The reported handwriting method transfers a learned grasp, collects IK writing demonstrations over the complete repertoire, fits one BC policy on the pooled data, and refines the composed policy with full-task outcome RL. Figure 2 shows this separation.

Component	Training condition	Training and transfer
Grasp controller	240 teacher episodes for BC, then pen-hold PPO under motion and contact	Transfer the selected learned grasp into the MLP
Writing data and BC	Traces, glyphs, words/cursive, and Chinese together	IK arm/wrist plus learned grasp supplies clean/noisy demonstrations; fit one all-task clone
Composed-policy RL	Full writing mixture with grasp and contact dynamics active	Both branches execute; refine outcomes while retaining grasp-mean weights

Table 4: Transferred grip controller, all-task writing demonstration collection and BC, and full-task composed-policy RL.

Recorded grasp lineage. The reported handwriting checkpoint transfers an existing grasp policy initialized from 240 teacher episodes and followed by 8M PPO steps over static hold, moving hold, and environment-driven scribble. The later trajectory-based implementation folds the blank-map encoder contribution and constant input entries into MLP biases.

All-task writing BC. The clone is fit once on the complete pool of traces, glyphs, words, cursive, and Chinese characters, and initializes final RL. IK supplies demonstrations but does not act during learned-policy evaluation. Figure 4 shows training progress for each stage.

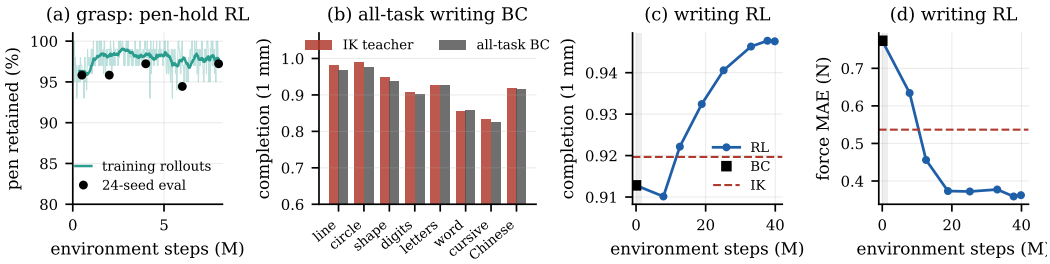


Figure 4: Training across the pipeline. (a) Pen-hold RL from the BC grasp: fraction of training episodes that keep the pen (thin: per iteration; thick: 15-iteration mean) and 24-seed evaluations over static hold, moving hold, and scribble. (b) Per-task completion of all-task BC and its IK teacher (50 seeds). (c, d) Writing RL completion and force MAE over environment steps; BC at step 0, IK dashed.

Outcome-based RL for writing. BC provides a broad initialization, but matching IK commands does not directly optimize the written trace or contact force. We therefore refine the writing branch with PPO while executing the full grasp–pen–arm system.

A.2 Detailed policy, imitation, and RL implementation

Trajectory-conditioned control. The input comprises 137 retained state entries (proprioception, tactile feedback, task and geometric state) and 16 future trajectory samples. Each sample contains tip-relative displacement, relative height, requested force, and nominal time offset. Samples are spaced by two trajectory indices. A tip-driven pointer advances through the request with a 1.5 mm proximity tolerance and a 0.25 s skip timeout; the request therefore does not advance solely according to wall-clock time. Pen-down input waypoints specify the sheet surface, rather than exposing IK's below-surface pressing offset. The policy thus receives explicit task geometry.

Separate grasp and writing features. Let $x_t \in \mathbb{R}^{217}$ be the normalized input and m select the arm, wrist, and stop dimensions. The Gaussian mean is

$$\mu(x_t) = (1 - m) \odot g_{\theta_g}(x_t) + m \odot w_{\theta_w}(x_t). \tag{3}$$

The grasp branch inherits the learned holding controller; the writing branch is initialized through imitation. The selected outputs form 34 physical controls and one stop entry, requesting termination above 0.5. A separate $217 \to 256 \to 256 \to 1$ critic has the same hidden activations. The implementation contains 383,082 parameters including inactive head rows and Gaussian log-standard-deviations. During RL, grasp-mean parameters and observation statistics remain fixed; log-standard-deviations remain trainable, including those for fingers.

Imitation initialization. IK supplies arm/wrist commands while a learned holding policy supplies finger commands. The collection recipe gathers clean rollouts and two noise-injected sets ($\sigma = 0.01, 0.02$ in executed action units), retaining clean IK labels. We use DART-style noise injection [17] with fixed variance. The delivered arm/wrist clone trains on the pooled demonstrations from all tasks and uses a weighted squared-error loss,

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}_{(x,a^*)} \sum_{d=1}^{9} w_d \big(\mu_d(x) - a_d^*/s_d\big)^2, \quad w_d = \mathrm{clip}\left[\left(\frac{\mathrm{median}_j\, \sigma_j}{\max(\sigma_d, 10^{-3})}\right)^2, 1, 100\right], \tag{4}$$

where s_d is the action scale and σ_d the demonstration-label standard deviation. The recorded recipe uses Adam, learning rate 3×10^{-4}, batch 256, and 60 epochs per candidate. The writing branch is merged with the grasp branch and the stop output is fitted on rollouts. The resulting all-task clone is the BC baseline below.

Rewarding drawing outcomes. The final reward configuration uses

$$r_t = \lambda_C \Delta C_t - \lambda_P \Delta P_t - \lambda_E \Delta E_t - \lambda_\tau \Delta \tau_t + \lambda_q \big(\|p_{t-1} - q_t\| - \|p_t - q_t\|\big) - D_t. \tag{5}$$

Here C_t is online ordered target coverage within 1 mm; P_t accumulates off-target ink normalized by expected pen-down samples, capped at 2; E_t is running contact-force MAE; and $\Delta\tau_t$ is elapsed time divided by nominal task duration. The guidance point q_t is the first uncovered glyph point at or after the coverage pointer, placed 1.5 mm below the sheet. The same q_t is used on both sides of the distance difference. D_t accounts for revoking accumulated coverage reward on a drop and charging remaining time for specified premature terminations. Coverage is an online surrogate for final alignment; RL has no IK-action or timing loss.

Table 5: Weights in the final drawing-outcome reward.

Weight	Value	Reward term
λ_C	1000	Ordered target coverage
λ_P	500	Off-target ink
λ_E	200	Contact-force MAE
λ_τ	300	Normalized elapsed time
λ_q	5000	Progress toward the guidance point

Stable policy refinement. We optimize the PPO clipped objective [18] using gradients accumulated over each full 512×1024 rollout. Up to four actor passes use Adam at 10^{-5} with linear decay. Proposed parameter displacements are rescaled if approximate KL on 16,384 randomly sampled rollout transitions exceeds 0.03; this is an empirical budget, not a hard KL guarantee. The critic receives two minibatch passes (batch 2048), after three initial critic-only iterations. We use $\gamma = 1$, GAE $\lambda = 0.95$, clip 0.2, reward scale 0.001, and initial arm/wrist noise $\exp(-4.6) \approx 0.01$. The recorded run collects 39.85M transitions in 193.8 minutes.

B Task catalogue, force-annotated strokes, and the 5D cursor

This section describes the task interface used by the reported system. It distinguishes the three holding tasks used for grasp learning from the seven writing tasks in the current suite and the three evaluation-only generalization tasks. The primary comparison used eight writing tasks, with cursive separate from word. Task geometry, target force, and trajectory timing jointly define the requested behavior. The current source catalogue is not a claim that archived checkpoints were retrained on every subsequent task change.

B.1 Tasks used in grasp and writing training

Task ID	Nominal duration	Requested behavior and configuration
L0: grasp training; arm/wrist motion supplied by the environment		
`L0_hold_static`	6 s	Retain the seated pen while the hand is still; no target ink strokes.
`L0_hold_moving`	8 s	Retain the pen during imposed transport and wrist disturbance: 11 waypoints, motion scale 2.2, arm-motion box half-extents $(0.12, 0.08, 0.085)$ m, wrist shake amplitude 4°.
`L0_scribble`	10 s	Retain the pen during an environment-driven paper-contact path: 14 waypoints, 3 mm commanded press depth, 8° wrist wobble, and 15 N impact limit. This is a contact-retention task, not glyph matching.
L1: geometric tracing		
`L1_trace_line`	8 s	A 60–120 mm straight line with randomized orientation and placement; flat force profile.
`L1_trace_circle`	10 s	A circle 40–90 mm in diameter; flat force profile.
`L1_trace_shape`	12 s	A sampled shape, 40–90 mm requested height: line, circle, square, triangle, spiral, sine, figure-eight, or zigzag; flat or ramp force profile.
L2: handwriting		
`L2_write_digits`	10 s	One digit, 50–65 mm requested height, with a recorded-pressure profile. Script-selected digits use the sans face in the current suite.
`L2_write_letters`	10 s	One capital letter, 50–65 mm requested height, sampled font, and recorded-pressure profile.
`L2_write_word`	$(8 + 3n)$ s	An n-character word from the content pool; requested height 20–35 mm, sampled font, slant 0 or 0.3, and recorded-pressure profile.
`L2_write_chinese`	12 s	One Chinese character from the at-most-six-stroke pool, requested height 50–65 mm, and recorded-pressure profile.
E2: evaluation-only content, excluded from the training task list		
`E2_write_word_unseen`	$(8 + 3n)$ s	One of 20 held-out English words, with the word-task geometry and force profile.
`E2_write_idiom`	48 s	One of ten four-character Chinese idioms composed of trained characters; character height 50–65 mm before width fitting.
`E2_write_chinese_unseen`	12 s	One of 122 characters outside the trained pool; every character has at least seven strokes, extending both content and stroke complexity.

Table 6: Task catalogue relevant to this paper, from the current samplers. Dimensions are requested geometric ranges before width fitting. Nominal duration paces the trajectory; it is not necessarily the final elapsed episode time. All listed L1/L2/E2 writing samplers use the pen configuration.

Figure 5 shows the retention tasks and one target per tracing and writing task.

Placement and variation. Stroke templates are normalized, scaled, optionally sheared, rotated, and translated on the sheet. The common placement routine samples rotation in $[-0.18, 0.18]$ radians unless overridden, respects a 30 mm margin, and reduces scale when needed to fit the width.

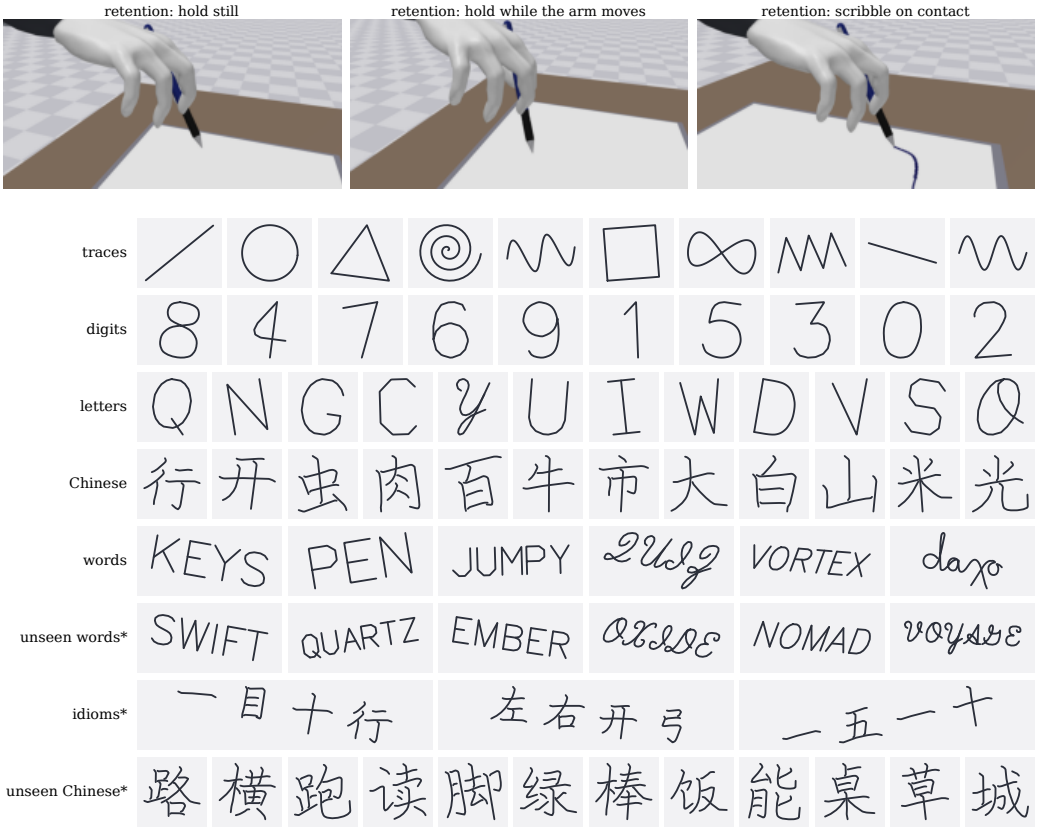


Figure 5: Task suite. Top: the three pen-retention tasks. Bottom: one target per tracing and writing task, drawn to scale with its size; * marks held-out content evaluated zero-shot.

Content, font, geometry, and force variation depend on the particular task; they should not all be assumed independent for every sampler. The current font pool contains block, sans, rounded sans, and script. Current content pools merge earlier training/test pools.

Nominal time versus episode horizon. The `duration` field sets the nominal trajectory budget. For writing with `end_mode=policy`, the implementation permits up to twice this nominal duration, with earlier termination possible through the policy's stop output, dropping, or applicable termination rules. Thus a word's $(8 + 3n)$ s configuration should not be interpreted as its measured writing time or as an unconditional hard timeout.

Additional registered tasks. The broader environment also registers finger-only scribbling and pen pickup at L0; brush strokes, Chinese brush writing, and brush layouts at L3; sketch hatching at L4; shape/rainbow/scene painting at L5; and chopstick holding, cylinder, cube, block, and sphere tasks at L6. These are outside the reported writing comparisons. The pen-writing idiom result uses the dedicated evaluation-only E2 sampler rather than the unrelated brush-layout task.

B.2 What a task configuration specifies

A task is sampled as a `TaskSpec`, rather than specified only by its natural-language instruction. Its ordered strokes, force annotations, metadata, and timing determine the actual control request.

Field	Type / unit	Meaning
`task_id`, `level`	Identifier	Select the task family and level.
`instruction`	Text	Human-readable description. The policy MLP does not encode this string as language.
`strokes`	List of $N_j \times 2$ arrays, m	Ordered centerline points in paper coordinates relative to the sheet center. List boundaries distinguish pen-down strokes.
`forces`	List of N_j arrays, N	Requested pressing force at each corresponding stroke point.
`widths`	List of N_j arrays, mm	Target mark widths derived from the force arrays and the tool's ink model.
`duration`	Seconds	Nominal schedule duration used to pace the requested strokes and transits.
`tool`, `start`	Enumerations	Tool identity and seated-grasp versus table-start initialization. The reported writing tasks use a seated pen.
`hold_motion`	Mode or null	Selects externally driven static/moving/scribble behavior for holding tasks.
`hold_scale`, `hold_waypoints`	Scale / count	Set imposed-motion amplitude and number of waypoints in holding tasks.
`hold_box`, `hold_jitter_deg`	m / degrees	Arm transport bounds and wrist-disturbance amplitude.
`scribble_depth_m`, `scribble_wrist_deg`	m / degrees	Commanded contact depth and wrist wobble for driven scribble.
`impact_limit_N`	N	Contact-impact limit used by the relevant task's success criterion.
`meta`	Dictionary	Sampled text, font, slant, shape, or other task-specific metadata.

Table 7: Task-definition fields. Reward weights, optimizer settings, and network architecture are separate from this schema. Stored legacy success thresholds should not be confused with the paper's reported 1 mm completion metric.

Force-annotated target strokes. For stroke j, write its configuration as

$$\begin{aligned} S_j &= \{(x_{j,i}, y_{j,i})\}_{i=1}^{N_j}, \qquad F_j = \{f_{j,i}\}_{i=1}^{N_j}, \\ W_j &= \{w_{j,i}\}_{i=1}^{N_j}, \qquad w_{j,i} = \mathrm{inkWidth}(\texttt{tool}, f_{j,i}). \end{aligned} \tag{6}$$

The force arrays are target commands, not measurements of the force the robot actually achieves. Their correspondence with stroke points makes force part of the requested writing content, alongside path and order. Width is derived from the same force values, rather than sampled as an unrelated visual decoration.

The default pen force-profile generator samples a base force between 0.9 and 2.5 N. A *flat* profile holds one sampled force along a stroke. A *ramp* modulates it monotonically along normalized arc length. A *recorded-pressure* profile samples a stored Wacom pressure curve, interpolates it along the stroke's arc length, and rescales it into 0.9–2.5 N. The general generator also supports a *swell* profile; it is not used by the listed L1/L2 samplers. All generated values are clipped to $[0.15, 4.0]$ N. In particular, the base-force range is not a strict bound on the ramp or swell profile before clipping.

From force to the target image and observed ink. The environment can rasterize a task map using

$$I^\star(x, y) \approx 255 \, \frac{f^\star(x, y)}{F_{\max}}, \qquad F_{\max} = 4\,\mathrm{N}, \tag{7}$$

with integer clipping to $[1, 255]$ on target pixels and zero background. Stroke thickness follows `widths`. The stored intensity is a force encoding; display software may invert it to render larger values as darker ink. Actual ink instead depends on simulated contact force and tip speed, so target intensity is not a guarantee of the deposited appearance. The current deposition model makes force affect width and darkness, with speed-dependent adjustments. Although these maps are useful for visualization and evaluation, the `traj2` actor consumes the numerical trajectory window, not map pixels.

B.3 The 5D cursor and its lookahead window

Absolute task-space request. The environment expands the ordered 2D strokes into a time-indexed 3D schedule containing approach, descent, pen-down motion, lifted transitions, and a final pen-up phase. Each requested sample can be represented as a five-dimensional cursor

$$c_k^\star = (x_k^\star, y_k^\star, h_k^\star, f_k^\star, t_k^\star). \tag{8}$$

Here x, y are horizontal target coordinates, h is height above the sheet, f is requested force, and t is nominal due time. "5D" therefore means position, force, and time; it is not a five-joint action or a five-DoF end-effector pose. The source stores these components across timeline/height/force arrays and constructs the five-entry representation for the observation.

For a pen-down request, exposed height is zero and force is positive. For a pen-up/transit request, force is zero and the requested height follows the lift trajectory. The teacher's 1.5 mm below-surface pressing target is not exposed as a negative target height: the task specifies surface contact and desired force separately. At overlapping strokes, the current implementation obtains timeline force through the spatial target-force lookup; it does not supply an additional stroke-specific force identifier to the actor.

Entry	Absolute request	Policy entry	Interpretation
1	$x_k^\star$	$x_k^\star - p_{t,x}$	Horizontal displacement, m
2	$y_k^\star$	$y_k^\star - p_{t,y}$	Horizontal displacement, m
3	$h_k^\star$	$h_k^\star - h_t$	Height error relative to the sheet, m
4	$f_k^\star$	$f_k^\star$	Requested force, N; not force error or measured contact force
5	$t_k^\star$	$t_k^\star - t$	Due-time offset, s; negative when the nominal due time has passed

Table 8: Semantics of one five-dimensional trajectory/cursor sample. Normalization is applied after assembling the policy vector.

What the policy actually sees. The actor observes a window, not just one current cursor. Let k_t be the current task pointer and K the last valid timeline index. With 16 points and stride 2,

$$k_{t,i} = \min(k_t + 2i, K), \qquad i = 0, \ldots, 15, \tag{9}$$

$$z_{t,i} = (x_{k_{t,i}}^\star - p_{t,x},\ y_{k_{t,i}}^\star - p_{t,y},\ h_{k_{t,i}}^\star - h_t,\ f_{k_{t,i}}^\star,\ t_{k_{t,i}}^\star - t). \tag{10}$$

The resulting $16 \times 5 = 80$ values are flattened and appended to the 137 retained state entries, yielding 217 inputs. At 25 Hz, adjacent samples are 0.08 s apart in nominal schedule time and the last is 1.2 s beyond the first. This is schedule spacing, not a guaranteed 1.2-second horizon relative to the current physical execution, because the pointer can lag. Near the end, indices are clamped to the last point. For holding tasks without a writing timeline, the trajectory window is zero.

Pointer advancement versus teacher timing. The reported configuration uses `adv_mode=tip`. A pen-down timeline point passes the advancement check when planar tip error is at most 1.5 mm and tip height is at most 0.8 mm above the sheet. The height condition is a geometric contact proxy, not a requirement that measured normal force exceed a threshold. Pen-up points use a 6 mm planar tolerance and a 15 mm vertical-error tolerance. If no point advances, the current point is skipped after the 0.25 s wait threshold. Skipping does not itself deposit ink or earn completion credit. Consequently, pointer progress, actual stroke coverage, and measured force remain distinct quantities.

C Extended results and diagnostics

C.1 Metric definitions and task-level effects

Ordered completion uses monotone alignment between the requested stroke sequence and deposited ink, with a 1 mm threshold. Precision is the fraction of ink samples within 1 mm of the target. Symmetric Chamfer distance measures geometric proximity, while the reported matched-stroke discrete Fréchet distance permits reversal when pairing and averages matched pairs. Force MAE compares eligible contact-force samples with the requested force. Elapsed time includes learned stopping or failure termination, and drop rate is the percentage of episodes ending in a detected pen drop.

C.2 Case study: RL suppressing stroke jitter

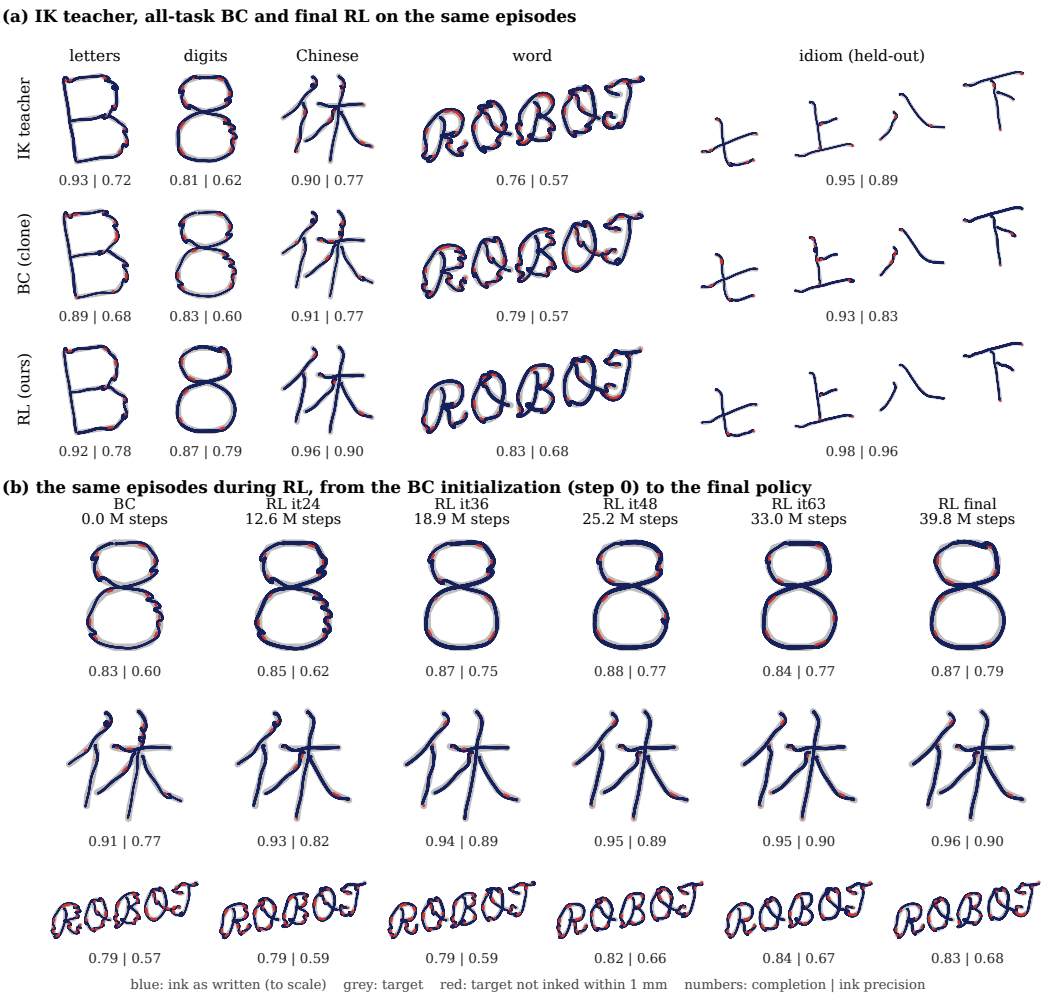


Figure 6: Matched episodes across IK, BC, and final RL (top), and their evolution from BC initialization through RL (bottom). Blue is deposited ink, gray is the target, and red marks target points not covered within 1 mm. Numbers report completion | ink precision. The high-frequency deviations in IK and BC leave off-target ink; RL produces smoother, more target-aligned strokes.

The gains extend across the full repertoire. Digits improve from 90.6% IK completion to 95.0%, Chinese from 91.7% to 96.4%, and words from 85.4% to 89.8%. The line is the exception: its completion is 97.5% for RL and 98.2% for IK. Seed 26254 produces no ink for both BC and RL after an initial tip excursion of roughly 50 mm and a hover termination. Empty ink receives a 50 mm Chamfer cap, increasing its influence on the aggregate. We retain this episode in all reported means.

C.3 Progress during final RL

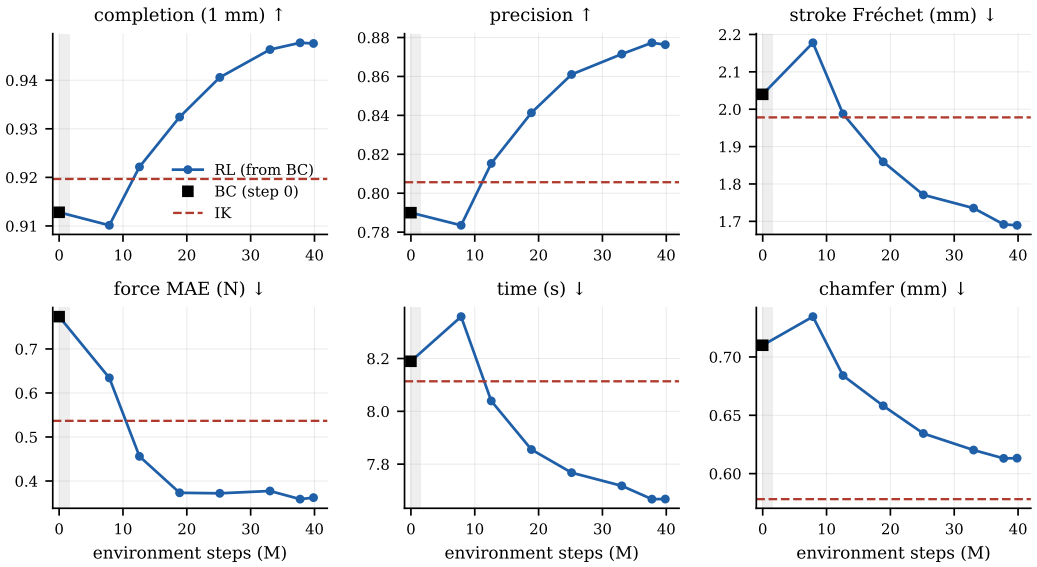


Figure 7: Fifty-seed evaluations during final RL. The initial point is BC and dashed lines indicate IK on the same seeds. Snapshot timing makes the horizontal axis approximate by at most one rollout. All checkpoints come from one training run.

The transition from imitation to RL is nonmonotone: early evaluations can lose completion or shape quality before later checkpoints improve completion, ink precision, force control, and elapsed time together. This motivates evaluating physical outcomes throughout refinement rather than treating a rising return as sufficient evidence of better writing.

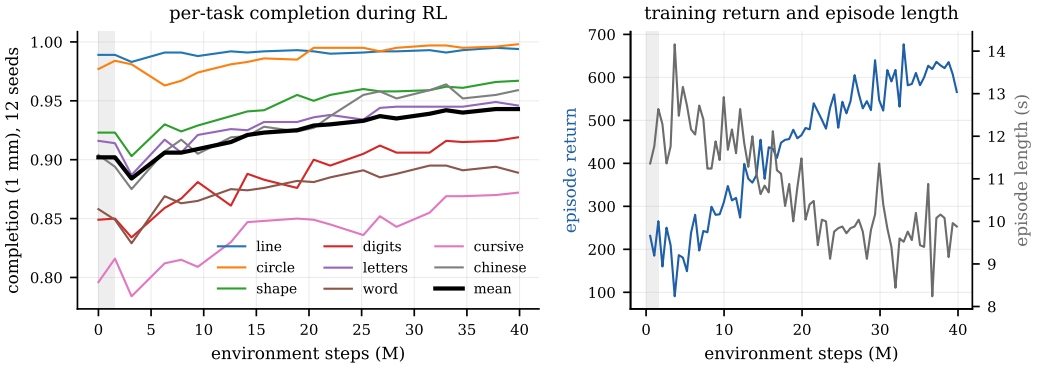


Figure 8: Twelve-seed task monitoring and training return/episode length during final RL. This monitoring set differs from the 50-seed aggregate evaluation and is descriptive rather than a repeated-training statistic.

The 12-seed BC monitoring mean is 90.2%, whereas the 50-seed BC mean is 91.3%. These baselines are not interchanged when quantifying improvement. Intermediate checkpoints are snapshots of one final RL run, not independent training seeds.